Order book performance

Mohit Kumar · Roll 10222 · mohit.24bcs10222@sst.scaler.com

AI-assisted work · Measured Linux timing, counters and CPU profiles

Measured results

C has the lowest replay median (82.038 ms), and the lowest average add and cancel times. BBO medians span 20.764–22.928 ns; their proximity to the timer overhead limits any useful ranking.

Metric	A: tree	B: vectors	C: one array
Total time (ms)	360.780	186.313	82.038
ns / add	344.624	136.811	95.418
ns / cancel	708.851	502.024	165.432
ns / BBO	22.928	21.579	20.764
IPC	0.905	0.914	1.854
Branch misses (%)	4.861	8.730	1.863
L1-dcache-load-misses	15,169,352	28,291,025	18,751,037
Cache misses	438,910	933,883	328,631
Page faults	17,028	22,748	16,354

Environment and correctness

The run used Ubuntu 20.04.6 in the Coder nami EC2/KVM guest: Xeon Platinum 8375C, 64 logical CPUs and about 124 GiB guest RAM. GCC 9.4 used C++17, -O2, debug symbols and frame pointers. Execution was pinned to CPU 63 at nice 10; VM scheduling and background load remain uncontrolled. Seed 42 produces 700,818 adds, 199,505 live cancels and 99,677 BBO queries, plus 10,000 top-ten depth pairs. All three designs and an independent reference agree at full depth after every event in the completed million-event Nami check. Mac stress, Clang and ASan/UBSan checks also pass.

What the measurements include

Timing cells are medians of five fresh-book runs; operation cells take the median of each run's average. Total replay includes depth/checksum work, excluding generation, construction, destruction and output. API timing uses a separate instrumented replay. Empty clock intervals are 19.3–19.5 ns and are not subtracted. steady_clock is monotonic; C++ mandates no particular OS API, CPU counter or physical resolution. Counter cells are medians of five whole-process perf runs; ratios are computed per run first. They include the privilege-drop helper, generation and teardown, so their scope is wider than the chrono interval. All nine requested events were available, user-only (:u), with 100% reported running time. There was no warmup and order was fixed A/B/C; A drifted from 309.518 to 373.579 ms, limiting broad causal claims.

Physical arrangement

A uses maps for IDs and price aggregates; B keeps ID history and two ascending vectors. C uses one 128-byte-aligned array: B0,A0,B1,A1,... . Even bid ranks descend; odd ask ranks ascend. The complete best records stay at slots 0/1, including after best removal. The 32-byte pair is tested within both 64- and 128-byte lines on stack, heap and populated vector relocation; this Xeon reports 64-byte lines. Tick-to-rank metadata is separate from the quote buffer. Prices remain integer ticks; decimal conversion is only for display.

Evidence: results/timings.csv; results/perf-stat-{A,B,C}.txt; results/perf-runs.csv; results/perf-report.txt. Method and hashes: results/measurement.md. Captured 16 October 2026. Workload hash 4920528738431414721.

Branches, instructions and IPC

A has the largest branch-miss count (12,960,810); B has the highest miss rate (8.730%). These are different rankings because A executes more branches. A searches ID and price trees; unpredictable comparisons and pointer loads are plausible costs. B's random cancels binary-search a growing history, which can explain a high miss fraction. C retires more instructions than B (1,110,701,807 versus 771,322,494) yet has higher process IPC (1.854 versus 0.914) and fewer misses. Instruction count alone does not explain latency. C's rank metadata avoids price search for existing-level updates; its absent-price search is ordinary lower_bound, not a branchless claim.

Cache behavior and page faults

B has the most generic cache misses (933,883), L1 load misses (28,291,025) and dTLB load misses (1,019,460). Its contiguous storage still reaches distant historical ID records on cancellation. C has fewer generic cache misses than A, but more L1 misses (18,751,037 versus 15,169,352); cache levels are not interchangeable. Page faults are also highest for B (22,748); A/C have 17,028/16,354. B retains 700,818 order slots for 501,313 live orders, while A/C remove cancelled IDs. Growing/touching storage is a plausible explanation; fault totals are not peak RSS, cache misses or proof of a particular allocation cause. C's fixed quote/rank payload is 160,000/40,000 bytes before hash storage and padding.

Where CPU samples landed

Nine total-only replays were sampled with cpu-clock:u at 999 Hz and frame pointers: 3,414/1,604/981 samples for A/B/C, with zero reported lost samples. A's replay has 63.50% of self samples, tree insertion 11.13%, and erase rebalancing 6.74%. B's replay/add_order account for 44.89%/34.10%. C's add_order has 30.48%, replay 26.61%, and hash rehashing 8.56%; its call graph places rehashing beneath addition. These are whole-recording sample shares, not measured API latencies. Inlining combines work under replay, and some libc addresses remain unresolved; those addresses are not assigned invented function names.

Production choice and next experiments

I would take C forward for the fixed tick range: its measured replay is fastest and its top-pair location is testable. It still allocates hash nodes and rehashes, and inserting/deleting a price level shifts one side and repairs ranks in O(L), creating a different tail risk. A adapts more simply to changing price bands. Before deployment I would measure p50/p99/max and memory high-water marks, reserve an explicit ID capacity, compare a pooled index, and reuse depth buffers. Steady-state churn, repeated best removal, sparse books and arbitrary ID arrival matter more than this growing workload alone. The approved admin profiler targeted only its child; setpriv ran project code as UID 1000. Paranoid remained 4, with no permanent capabilities or security changes.

Brief: Capstone: Build a Low-Latency Order Book. README explains the interfaces and method; the student-authorship condition still requires review.